@Override, Overload, Polimorfismo y Excepciones

Programador Universitario en Sistemas
Primer Año - 2° cuatrimestre

En esta clase vamos a trabajar:

- Sobreescritura de métodos (@Override).
- Sobrecarga de métodos y constructores.
- Diferencias entre sobreescritura y sobrecarga.
- Polimorfismo.
- Tipo declarado y tipo real de un objeto.
- Binding dinámico.
- Referencias polimórficas.
- Excepciones.
- Jerarquía de excepciones.
- Excepciones como objetos.
- try, catch y finally.
- Propagación de excepciones.
- throw y throws.
- Excepciones checked y unchecked.
- getMessage() y printStackTrace().
- Encadenamiento de excepciones.
- Buenas prácticas en el manejo de excepciones.

¿Qué pasa cuando una subclase necesita cambiar un comportamiento?

Animal.java

```java
package universidad.modelo;

public class Animal {

    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void hacerSonido() {
        System.out.println("El animal hace un sonido");
    }
}
```

Y una clase específica:

Perro.java

```java
package universidad.modelo;

public class Perro extends Animal {

    public Perro(String nombre) {
        super(nombre);
    }
}
```

Queremos que:

Animal → sonido genérico

Perro → "Guau"

Gato → "Miau"

Ahí aparece **@Override**.

¿Qué es Override?

Override significa que una subclase proporciona una nueva implementación para un método heredado.

En Animal:

```
public void hacerSonido() {
    System.out.println("El animal hace un sonido");
}
```

En Perro:

```
@Override
public void hacerSonido() {
    System.out.println("Guau");
}
```

¿Dónde se utiliza?

Main.java

```
Perro perro = new Perro();

perro.hacerSonido();
```

Resultado:

Guau

@Override permite además que el compilador controle que realmente estamos sobrescribiendo un método heredado.

Regla

Override

→ herencia

→ mismo método

→ misma firma

→ nueva implementación

Overload

Overload = sobrecarga.

Tenemos varios métodos con el mismo nombre, pero diferente lista de parámetros.

```
public int sumar(int a, int b) {
    return a + b;
}

public double sumar(double a, double b) {
    return a + b;
}
```

Desde Main:

```
calc.sumar(2, 3);       // sumar(int, int)

calc.sumar(2.5, 3.5);   // sumar(double, double)
```

Java decide cuál utilizar según los argumentos.

¿Qué NO es Overload?

No podemos hacer:

```
public int calcular(int numero) {
    return numero * 2;
}

public double calcular(int numero) {
    return numero * 2.0;
}
```

¿Por qué?

Los parámetros son iguales:
calcular(int)
calcular(int)
Cambiar solamente el tipo de retorno no alcanza para sobrecargar.

Override vs Overload

Override	Overload
Requiere herencia	No necesariamente
Mismo nombre	Mismo nombre
Misma firma	Diferentes parámetros
Redefine comportamiento	Agrega otra variante
Se relaciona con ejecución	Se resuelve en compilación

Para recordar
Override → redefinir

Overload → sobrecargar

Polimorfismo

El polimorfismo es la capacidad de utilizar una misma referencia o interfaz de programación para trabajar con objetos de diferentes tipos concretos.
La idea central es:

Una misma operación puede tener comportamientos diferentes dependiendo del objeto sobre el que se ejecuta.
En POO, esto se logra principalmente mediante:

Herencia + Override + Referencia de tipo general

Supongamos un sistema de pagos:



Ambos pueden tener:

```
public void pagar(double monto)
```

Pero cada uno implementa la operación de manera diferente.

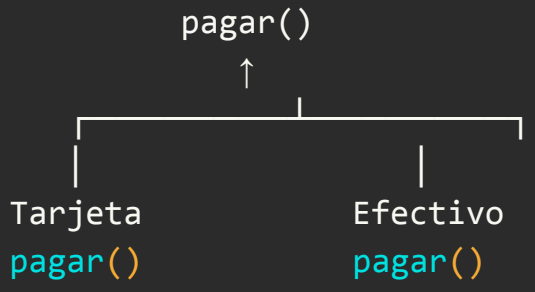
¿Por qué es útil el polimorfismo?

Podemos trabajar de forma general:

```
MedioPago medio = obtenerMedioPago();

medio.pagar(10000);
```

El código que utiliza MedioPago no necesita conocer si recibió una Tarjeta o Efectivo.



La ventaja fundamental

Podemos escribir código que trabaje con el **tipo general** y dejar que cada clase concreta determine su comportamiento.

Esto permite:

- reutilizar código;
- reducir dependencias entre clases;
- agregar nuevas implementaciones sin modificar el código general;
- construir sistemas más extensibles.

Polimorfismo + Override

Si tenemos:

```
Animal
public void hacerSonido() {

System.out.println("Animal");
}

Perro
@Override
public void hacerSonido() {
    System.out.println("Guau");
}
```

Entonces:

```
Animal animal = new Perro();

animal.hacerSonido();
```

imprime:

```
Guau
```

Aunque la referencia sea Animal, se ejecuta la implementación de Perro.

Binding dinámico

La elección de la implementación sobrescrita se realiza teniendo en cuenta el tipo real del objeto durante la ejecución.

```
Animal animal = new Perro();

animal.hacerSonido();
```

Conceptualmente:

```
animal
  ↓
objeto Perro
  ↓
Perro.hacerSonido()
```

Esto se conoce como binding dinámico.

¿Qué es el Binding (Enlace)?

Es el proceso que asocia la llamada a un método con el código real que se va a ejecutar.

Binding Dinamico (o tardío):
Ocurre durante la ejecución del programa. El sistema busca el tipo real del objeto en memoria (Perro) en lugar de guiarse por el tipo de la variable (Animal).

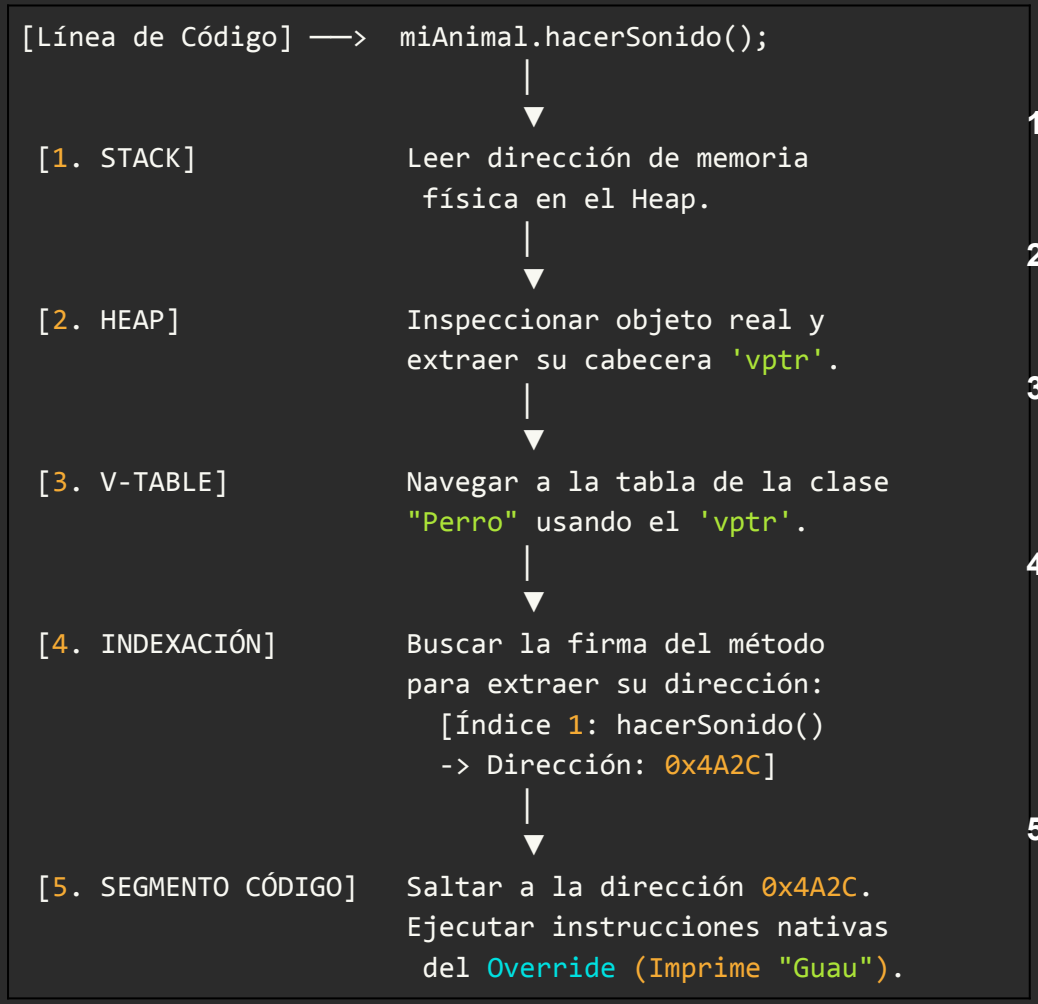
Arquitectura de Memoria y Desplazamiento Estático



[Virtual Method Invocation en Manning]

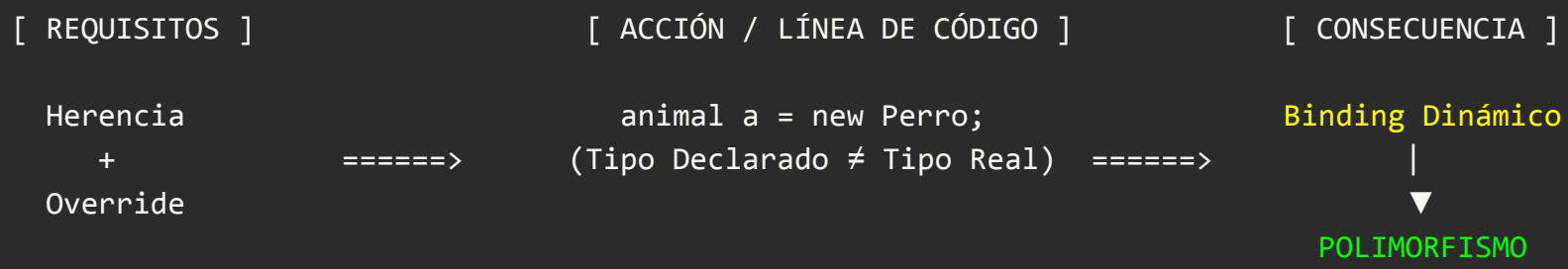
[Memory Layout of Objects in Java en Baeldung]

[Documentación de la Especificación de la Máquina Virtual de Java (JVM)]



1. **Desreferenciación:** El procesador lee el puntero alojado en el *Stack* para ubicar la dirección base del objeto en el *Heap*.
2. **Extracción del `vptr`:** Accede a la cabecera del objeto `Perro` y extrae el valor de su puntero `vptr`.
3. **Indexación de la V-Table:** El hilo sigue el `vptr` hasta la tabla de la clase correspondiente y busca por índice el método solicitado.
4. **Resolución del Override:** La V-Table de `Perro` contiene la dirección `0x4A2C` (que apunta al código específico de la clase hija). *Nota: Si Perro no hubiera hecho override, la tabla apuntaría por herencia a la dirección del padre.*
5. **Invocación Final:** Se realiza un salto de ejecución (*Jump*) a la sección de código ejecutable, procesando la lógica específica.

La cadena completa



Idea fundamental

Una misma operación puede producir comportamientos diferentes dependiendo del objeto concreto.

Polimorfismo en Colecciones

Almacenamiento Genérico

Podemos agrupar diferentes objetos en un único array usando la clase padre común:

```
Animal[] animales = {
    new Perro(),  // Cada objeto mantiene
    new Gato(),   // su comportamiento
    new Perro()   // específico
};
```

Procesamiento Unificado

Recorremos la colección enviando el mismo mensaje a todos. El binding dinámico hace el resto:

```
for (Animal animal : animales) {
    animal.hacerSonido();
}
```

Resultado en Consola:

Guau

Miau

Guau

El Beneficio Arquitectónico

¿Por qué este código es tan potente?

Desacoplamiento Absoluto

El bucle for trabaja exclusivamente con la clase Animal.
No necesita conocer, preguntar, ni validar qué clase concreta (Perro, Gato, etc.) está procesando en cada iteración.

Escalabilidad Total (Código Extensible)

Si mañana añadimos al sistema las clases Vaca, León o Loro, este bucle seguirá funcionando exactamente igual.

Podemos expandir el programa agregando nuevos animales sin modificar una sola línea de la lógica de reproducción de sonido.

Un programa puede compilar y fallar

Mitos y Realidades en Ejecución

```
int resultado = 10 / 0;
```

Instancia	¿Qué dice Java?	Estado
Tiempo de Compilación	*"La sintaxis es válida, todo en orden."*	**Aprobado** (Compila)
Tiempo de Ejecución	*"¡Imposible dividir por cero!"*	**Explotó**

Conclusión: Cuando las reglas de la sintaxis se cumplen pero la lógica del mundo real falla, Java genera una **Excepción** (en este caso, ArithmeticException).

Tipos de errores

Podemos distinguir:

Error de compilación

```
int numero = "Hola";
```

El compilador no permite generar correctamente el programa.

Error lógico

```
int resultado = precio - descuento;
```

cuando la lógica requería otra operación.

Error durante la ejecución

```
10 / 0
```

El programa llega a una situación problemática mientras se ejecuta.

¿Qué es una excepción?

¿Qué es una excepción?
Es un objeto que se crea en memoria cuando ocurre un error.

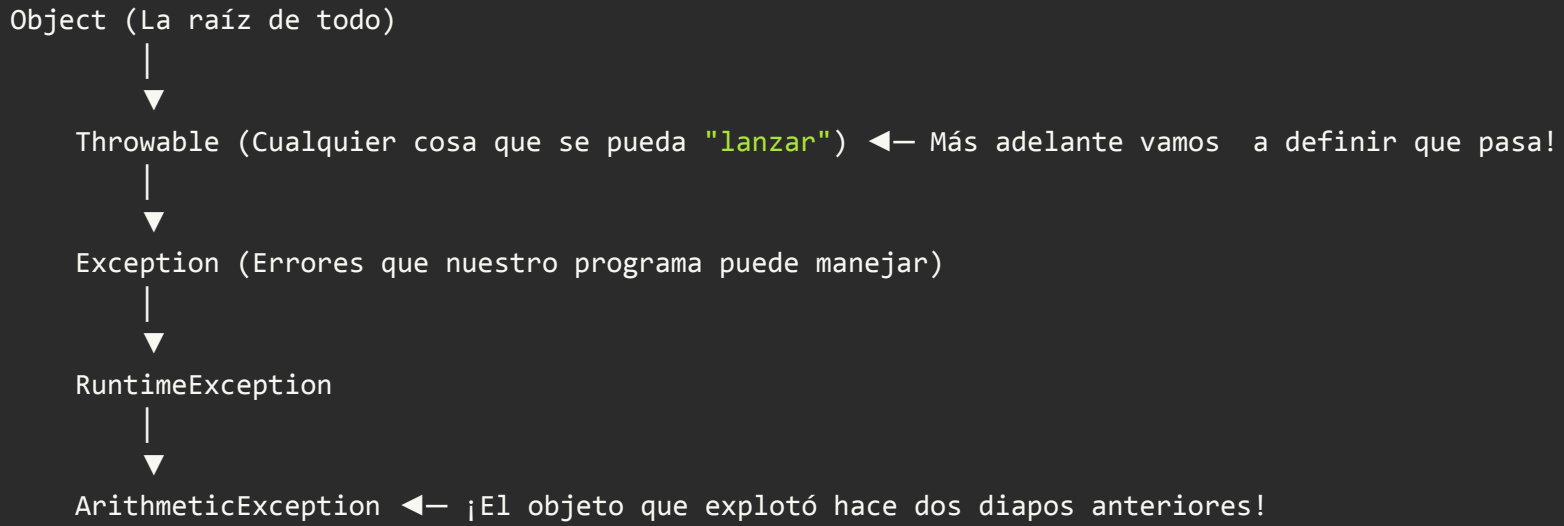
¿De dónde sale ese objeto?
Se instancia a partir de una clase específica (como ArithmeticException).

¿Cómo se organizan?
Forman parte de una estructura de herencia. Todas las excepciones de Java descienden de una superclase común.

¿Qué es una excepción?

Una excepción representa una situación anormal que ocurre durante la ejecución.
Ejemplos:

ArithmeticException
NullPointerException
ArrayIndexOutOfBoundsException



Ejemplos

ArithmeticException

```
public class Main {

    public static void main(String[]
args) {

        int resultado = 10 / 0;


System.out.println(resultado);
    }
}
```

Se produce:
ArithmeticException

La ejecución normal se interrumpe en la operación problemática.

NullPointerException

```
Persona persona = null;
```

Y luego:

```
persona.getNombre();
```

La referencia:

```
persona
```

no apunta a ningún objeto.

Por eso Java genera:
NullPointerException

ArrayIndexOutOfBoundsException

```
int[] numeros = {10, 20, 30};

System.out.println(numeros[10]);
```

Los índices válidos son:

0
1
2

Intentamos acceder al índice:

10

Se produce una excepción

¿Qué pasa si no manejamos la excepción?

```java
public static void main(String[] args) {

    int resultado = 10 / 0;

    System.out.println("Fin");
}
```

Interrupción: La excepción rompe el flujo normal del programa de forma inmediata.

Consecuencia: Por eso, la instrucción System.out.println("Fin"); jamás llega a ejecutarse.

El Reporte: Java detiene todo y muestra la información del error en la consola (Stack Trace):

```
Exception in thread "main" java.lang.ArithmeticException: / by zero at
Main.main(Main.java:3)
```

¿Podemos manejarla?

Java nos proporciona una estructura nativa para intentar ejecutar acciones propensas a fallar y atrapar el error antes de que rompa el programa:

```
try
catch

Ejemplo:

try {

    int resultado = 10 / 0;

} catch (ArithmeticException e) {

    System.out.println("No se puede dividir por cero");
}
```

try (Intentar): Envuelve las líneas de código peligrosas que podrían generar un fallo en tiempo de ejecución.

catch (Atrapar): Intercepta la excepción específica declarada entre paréntesis y ejecuta un plan de contingencia.

try - catch

try {

Dentro del try colocamos el código que puede producir una excepción que queremos manejar.

```
try {

    int resultado = 10 / 0;

}
```

Podemos interpretarlo como:

"Intentá ejecutar este código."

Si ocurre una excepción compatible con un catch, se pasa al bloque correspondiente.

```
try {

    int resultado = 10 / 0;

} catch (ArithmeticException e) {

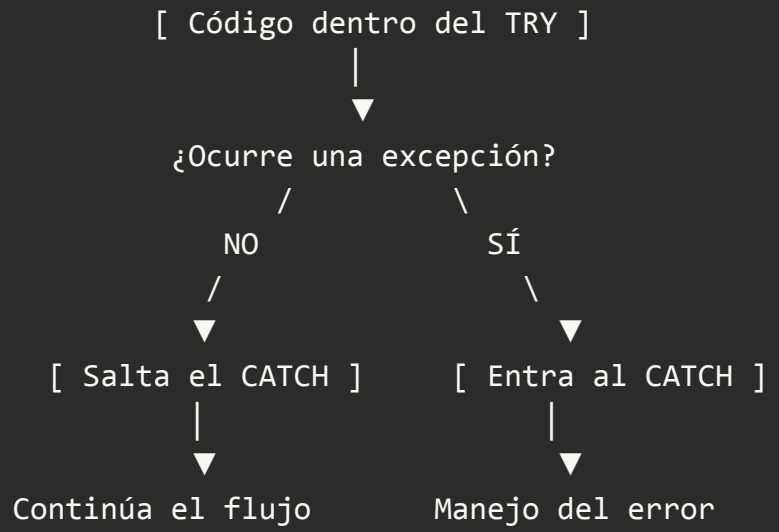
    System.out.println("No se puede dividir por cero");
}
```

El catch recibe la excepción:

ArithmeticException e

y permite ejecutar un tratamiento alternativo.

Flujo de try/catch

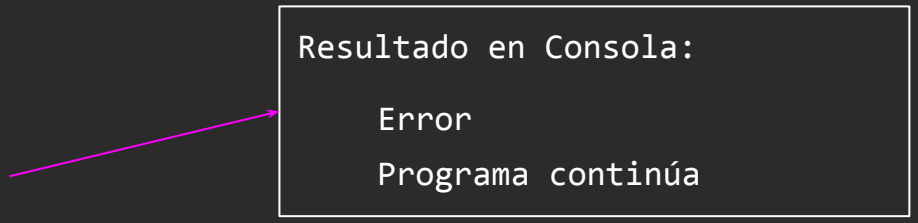


Si el código del try funciona perfectamente y **no ocurre ninguna excepción**, el bloque catch **se ignora por completo** y no se ejecuta.

¿Qué pasa después?

El Flujo Sigue Vivo

```
try { int resultado = 10 / 0;
}
    catch (ArithmeticException e)
{
    System.out.println("Error");
}
    System.out.println("Programa continúa");
```



El bloque catch actuó como un escudo: manejó la situación crítica, evitó que el programa muriera de forma abrupta y permitió que el software **continuara con el flujo posterior** normalmente.

La excepción también puede propagarse

Supongamos:

```
public static int dividir(int a, int b) {
    return a / b;
}
```

Desde main:

```
int resultado = dividir(10, 0);
```

La excepción aparece dentro de:
dividir()

pero puede propagarse hacia:
main()

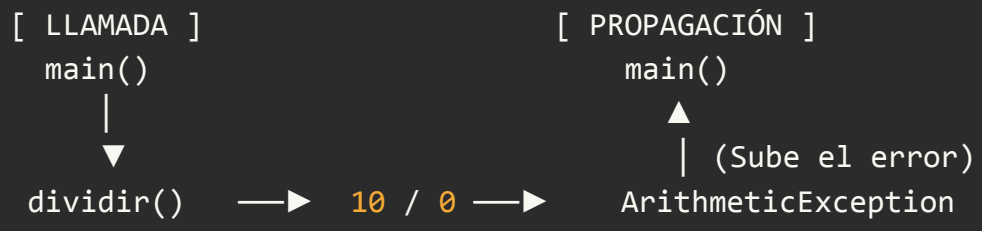
si no es manejada allí.

Cuando un método (dividir) explota y no tiene un bloque try-catch, no se guarda el problema para sí mismo: le "**pasa la pelota**" al método que lo llamó (main). Si nadie la atrapa en toda la cadena de llamadas, el programa se detiene. A esto se le conoce en programación como **desapilar la pila** (*Stack Unwinding*).

Propagación

Ciclo de la Excepción (La Pila)

El error viaja hacia abajo en las llamadas, pero sube si nadie lo atrapa:



La Solución: Captura en el origen (main)

Atrapamos la excepción en el método que inició la llamada para salvar el programa:

```
try { dividir(10, 0);
}
  catch (ArithmeticException e)

{ System.out.println("Error");
}
```

Cuando ocurre una excepción, Java busca un bloque try-catch en el método actual (dividir). Si no lo encuentra, destruye el contexto de ese método y **"desapila"** el error hacia el método anterior (main). Hay dos opciones como arquitecto de software: **manejarlo apenas ocurre**, o **dejar que suba** para centralizar los errores en un punto más alto del programa.

throw

No siempre debemos esperar a que Java explote por un error matemático o de memoria. Nosotros, como programadores, podemos **provocar y lanzar una excepción de forma intencional** cuando se rompe una regla de negocio.

```
public static void validarEdad(int edad)
{
     if (edad < 0) { // Creamos y lanzamos el objeto de error manualmente

               throw new IllegalArgumentException("La edad no puede ser negativa"); }
}
```

throw: Palabra clave que significa **"Lanzar explícitamente una excepción"** al sistema.

new ...: Instancia un **objeto** de error en ese mismo instante, interrumpiendo el flujo.

Para Java, que un entero (int) valga -5 es perfectamente válido y compila sin problemas. Sin embargo, en el mundo real, una persona no puede tener edad negativa. Usar throw nos permite **elevar una falla de lógica comercial al rango de un error de sistema**, blindando el programa contra datos corruptos antes de que causen estragos en la base de datos.

throw vs throws

throw ⟶ La Acción (El disparo)

Se usa **dentro** del cuerpo del método para ejecutar el lanzamiento de la excepción. Usa la palabra en singular porque dispara **un solo objeto** de error a la vez

```
throw new IllegalArgumentException();
```

Ubicación: Dentro del método.
Acompañado por: Un objeto creado con new.

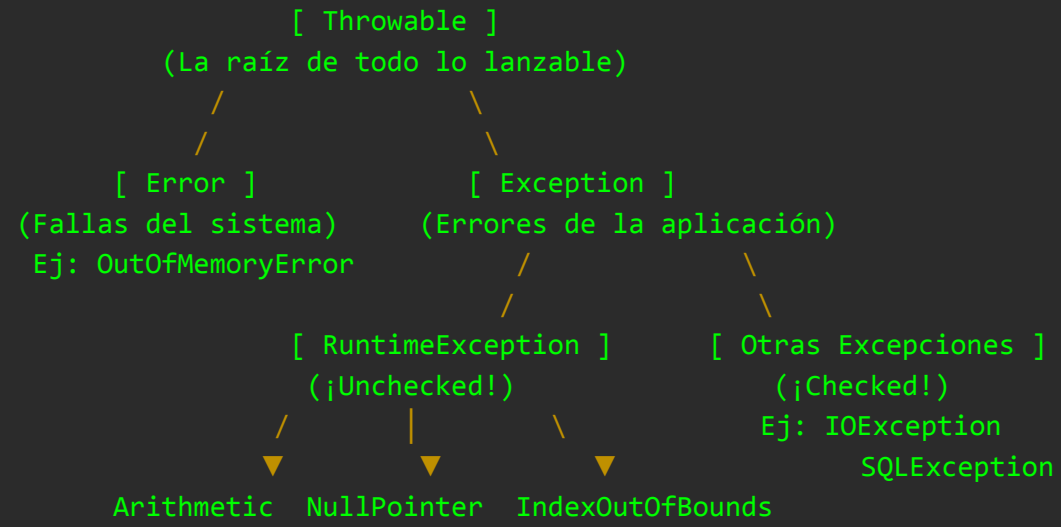
throws ⟶ El Anuncio (El Cartel de Advertencia)

Se usa en la firma del método para avisar al compilador y a otros programadores que este método es peligroso y puede propagar errores. Como es en plural, te permite listar múltiples clases de excepciones separadas por comas

```
public void leerArchivo() throws IOException,
SQLException { // ... código que puede fallar }
```

Ubicación: En la firma/declaración del método.
Acompañado por: Una o más clases de excepciones (sin el new).

Jerarquía de excepciones



Throwable: Es la superclase del reino. Si no hereda de aquí, Java no lo puede lanzar con un throw.
Error: Problemas críticos de hardware o de la Máquina Virtual (JVM). Tu programa no debería intentar capturarlos (si te quedas sin memoria, el programa muere).
Exception: El terreno de juego de uds como programadores. Son los errores que sí podemos prever, atrapar y solucionar en nuestro código.

¿Todas las excepciones se manejan igual?

Las excepciones que heredan de **RuntimeException** son **Unchecked** (no verificadas): ocurren por errores del programador (como un NullPointerException) y el compilador no te obliga a usar try-catch.

En cambio, las **Otras Excepciones** (que heredan directo de Exception) son **Checked** (verificadas): representan factores externos (un archivo que no existe, la base de datos caída) y Java **los obliga (programadores) de forma estricta** a manejarlas con try-catch o declararlas con throws antes de dejarte compilar.

finally

El bloque finally es el **"pase lo que pase, esto se hace"** de Java. No importa si el código del try funcionó a la perfección o si saltó una excepción que nos mandó al catch: **las instrucciones dentro de finally se van a ejecutar siempre.**

```
try { //Código peligroso (Ej: Abrir un archivo)
}
catch (Exception e) { // Plan de contingencia (Si el archivo no existía)
}
finally { // Se ejecuta al final, pase lo que pase en el try o catch.
}
```

EJEMPLO: “Garantía de Limpieza en Memoria”

Cuando tu programa se conecta a una Base de Datos, abre un Archivo de Texto o inicia un Socket de Red, está consumiendo canales del sistema operativo. Si ocurre un error a mitad de camino y el programa salta al catch, esa conexión podría quedar abierta para siempre (provocando un goteo de memoria o Memory Leak). finally es el lugar sagrado donde ponemos los métodos .close() para asegurar que el sistema devuelva los recursos, pase lo que pase.

Múltiples bloques catch

Un solo bloque try puede albergar código que falle por **diferentes motivos** (un error matemático, un archivo que no está, o un objeto que apunta a null). Java nos permite encadenar varios bloques catch para asignar **un escudo específico a cada tipo de problema.**

```
try {

} catch (ArithmeticException e) {
    // Solo se activa si la falla es matemática
    System.out.println("Problema matemático");
} catch (NullPointerException e) {
    // Solo se activa si un objeto es null
    System.out.println("Referencia null");
}
```

Java recorre los `catch` de arriba hacia abajo. **Solo se ejecutará el primero** que coincida exactamente con la excepción lanzada. Los demás se ignoran.

Cuidado con el Orden de Herencia!

Cuando se apilan múltiples bloques catch, debemos ir **siempre de lo específico a lo genérico**. Si colocamos un catch (Exception e) al principio de la lista, ese bloque atrapará absolutamente todo debido al polimorfismo, provocando que los catch de abajo (ArithmeticException, etc.) se vuelvan inaccesibles. El compilador de Java detectará este "código inalcanzable" y arrojará un error antes de dejarte ejecutar el programa.

Excepciones Personalizadas: Extendiendo el Lenguaje

Java nos da cientos de errores listos para usar, pero no sabe qué es una "Edad Inválida" o un "Saldo Insuficiente". Creando nuestras propias clases de excepción, logramos que los errores hablen **el mismo idioma que las reglas de nuestro negocio.**

```
// 1. HERENCIA: Nos colgamos del árbol genealógico oficial de Java
public class EdadInvalidaException extends Exception {

// 2. CONSTRUCTOR: Definimos cómo se crea nuestro error personalizado
public class EdadInvalidaException(String mensaje) { // 3. POLIMORFISMO: Le pasamos el mensaje a la superclase (Exception)
super(mensaje); }
}
```

extends Exception: Convierte a esta clase común y corriente en una Checked Exception oficial.
super(mensaje): Invoca al constructor de la clase padre para que guarde el texto del error en la memoria del sistema.

getMessage()

getMessage() permite obtener el mensaje asociado a la excepción.

```
try {

    throw new IllegalArgumentException(
        "La edad no puede ser negativa"
    );

} catch (IllegalArgumentException e) {

    System.out.println(e.getMessage());
}
```

Resultado:

La edad no puede ser negativa

Es útil cuando queremos mostrar al usuario o registrar una explicación concreta del problema.

Utilizando nuestra excepción

```
public static void validarEdad(int edad) throws
EdadInvalidaException {
      if (edad < 0) {
          //Lanzamos manualmente nuestra excepción
personalizada
              throw new EdadInvalidaException("La edad no
puede ser negativa"); }
}

// ——— Desde el método Main: ———————//

try { validarEdad(-5); // Ejecución peligrosa }

catch (EdadInvalidaException e) { // CAPTURA: Atrapamos el
error y extraemos el mensaje original

System.out.println(e.getMessage());
}
```

El ecosistema de control en acción:

Aquí se integran los tres roles clave de Java: **throw** detecta la falla y dispara el objeto;

throws advierte formalmente el peligro en la firma del método; y el **try-catch** ataja el impacto en el main.

Además, **e.getMessage()** demuestra el polimorfismo recuperando el texto que viaja encapsulado dentro del error.

printStackTrace()

printStackTrace() muestra información detallada de la excepción y la secuencia de llamadas que llevó hasta el error.

```
try {

    dividir(10, 0);

} catch (ArithmeticException e) {

    e.printStackTrace();
}
```

Podemos obtener información similar a:

```
ArithmeticException: / by zero
    at Calculadora.dividir(...)
    at Main.main(...)
```

Esto resulta especialmente útil para diagnosticar dónde se produjo el problema.

getMessage() vs printStackTrace()

Método	¿Qué muestra?
getMessage()	El mensaje asociado a la excepción
printStackTrace()	Información de la excepción + recorrido de llamadas

```
System.out.println(e.getMessage()); // información puntual.

e.printStackTrace(); // información detallada para analizar el problema.
```

Buenas Prácticas en el Manejo de Excepciones

La Especificidad ante todo

No usar una red gigante para atrapar un pez pequeño. Ve directo al grano.

- **Evitar:** catch (Exception e) sin un motivo de peso. Oculta errores inesperados.
- **Preferir:** catch (NumberFormatException e) cuando sabes exactamente qué vas a controlar.

Control de Flujo: if/else vs. Excepciones

Las excepciones son para eventos **anómalos e inesperados**, no para la lógica cotidiana del programa.

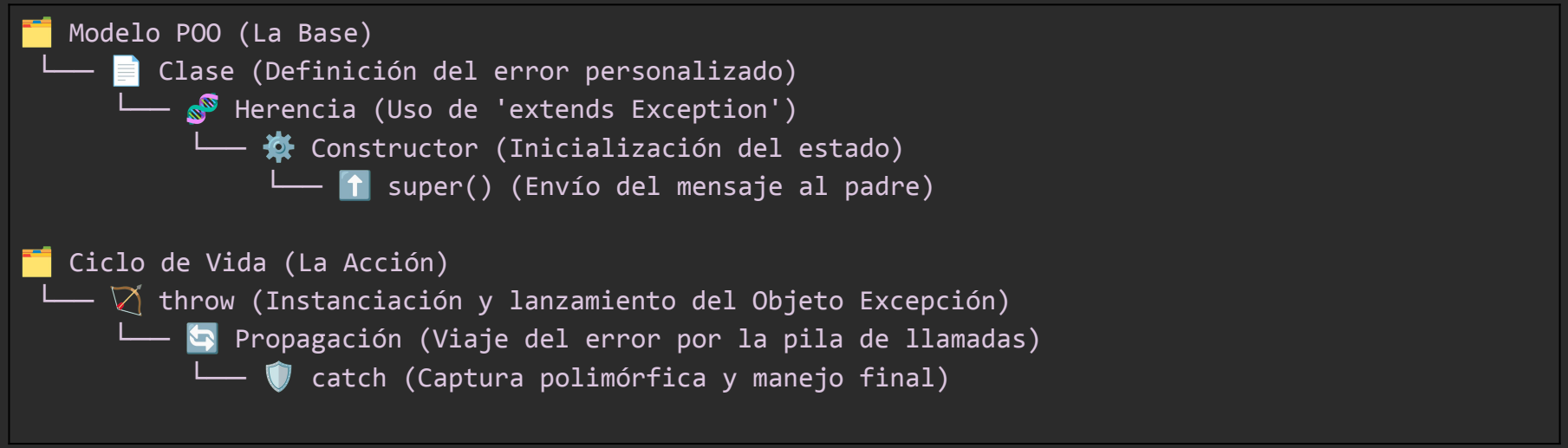
- **Mal uso:** Lanzar un error para validar si un usuario completó un campo obligatorio.
- **Buen uso:** Utilizar estructuras if / else para condiciones normales y esperables.

Mensajes con Contexto Real

Un mensaje de error debe guiar al programador o al usuario a la solución, no ser un misterio.

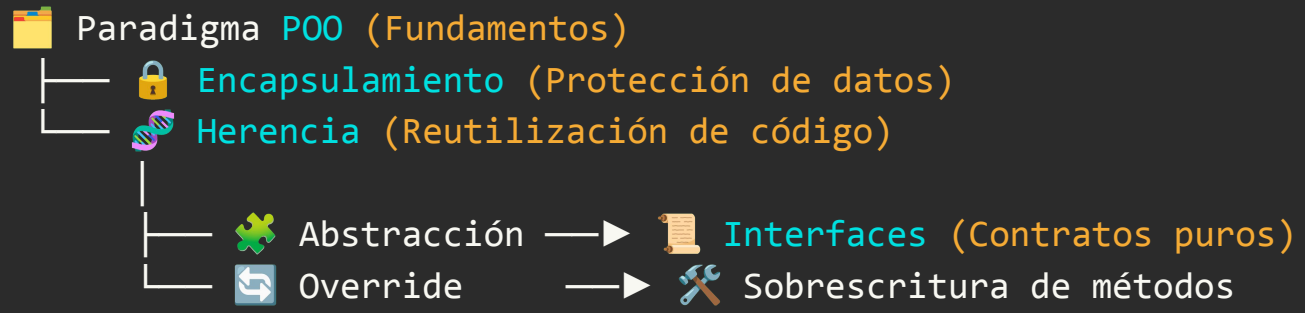
- **Insuficiente:** "Error" o "Fallo en el sistema".
- **Informativo:** "Edad inválida: -5 (El valor debe ser mayor a 0)".

Integración: ¿qué estamos haciendo?



Cuando lanzas un error, estás aplicando **Polimorfismo, Herencia y Encapsulamiento** al mismo tiempo. Java maneja las tragedias de tu código usando exactamente las mismas reglas con las que creas un `Perro` o un `Gato`.

El Mapa Genético de la POO



Este mapa demuestra que el **Polimorfismo** no existe en el vacío. Requiere obligatoriamente de la **Herencia** (o implementación de **Interfaces**) para crear la estructura de clases, y del **Override** para que cada objeto tenga su propia versión del comportamiento. El **Binding dinámico** es simplemente el mecanismo que usa la Máquina Virtual de Java para unir todas estas piezas en milisegundos cuando el programa está corriendo.

Los conceptos empiezan a conectarse

No estudiamos cada tema como una isla separada; construimos conocimiento agregando capas de complejidad sobre una misma base sólida:

🗂️ Nivel 1: Los Ladrillos Fundacionales
└── 📄 Clases y Objetos (La plantilla y la memoria)

🗂️ Nivel 2: Estructura y Flexibilidad
└── 🧬 Herencia y Abstracción (Líneas familiares y contratos)

🗂️ Nivel 3: Comportamiento Inteligente
└── 🔄 Override y Polimorfismo (Especialización y dinamismo)

🗂️ Nivel 4: Robustez del Sistema
└── 🛡️ Excepciones (Estructuras de POO aplicadas al control de fallas)

BIBLIOGRAFÍA

Bibliografía y fuentes de consulta

- **Aguilar, L. J.** — *Java: Programación Orientada a Objetos*.
- **Martínez, I. Z., Aguilar, L. J.** — *Estructuras de Datos en Java*.
- **Oracle** — *Java Documentation*. Documentación oficial del lenguaje Java.
- **Material teórico y práctico de la cátedra** — Teorías y prácticas previas de Programación Orientada a Objetos.